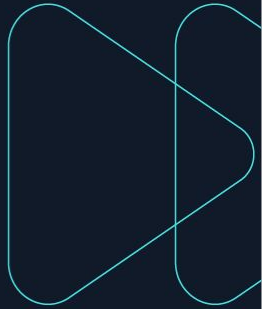


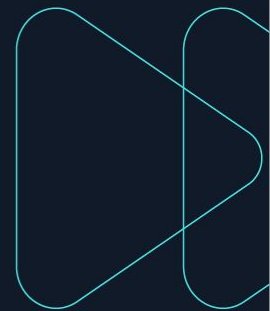
MACHINE LEARNING CLASIFICACIÓN Y REGRESIÓN



Aprendizaje supervisado

Módulo 3

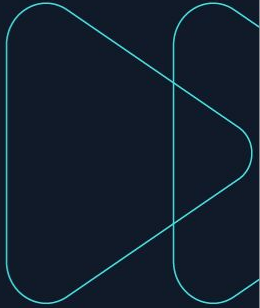
Unidad 2: Vectores de soporte



Unidad 2

- Árboles de decisión
- Random Forest
- Máquina de vectores de soporte
- Regresión por vectores de soporte

Arboles de decisión



Arboles de decisión

Es un modelo fundamental en aprendizaje automático que **divide los datos en subconjuntos** mediante reglas binarias simples.

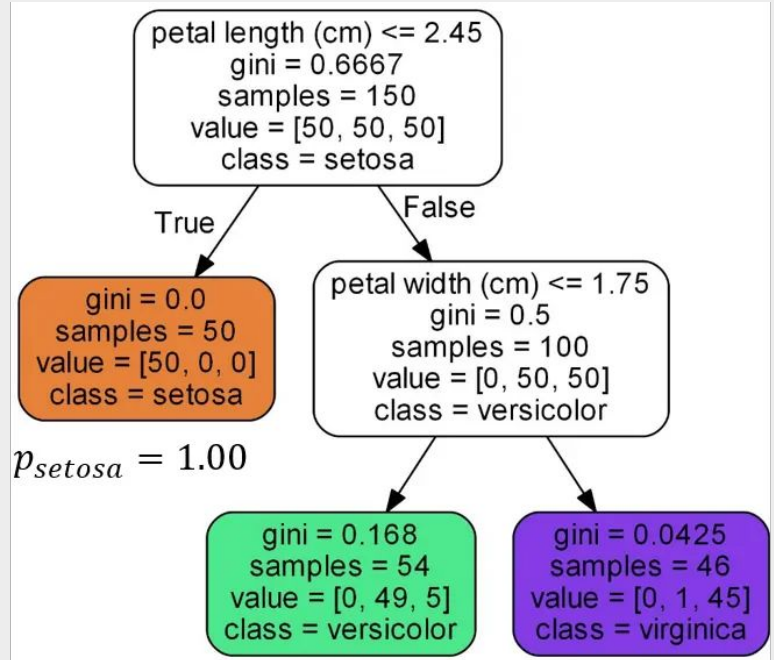
El objetivo del algoritmo es construir subconjuntos donde los datos sean lo más homogéneos posible.

Para medir esa "pureza" se utilizan funciones de impureza.

Estos modelos sirven tanto para problemas de clasificación como regresión, ofreciendo interpretabilidad y flexibilidad.



Arboles de decisión



$$p_{setosa} = 1.00$$

$$p_{versicolor} = 0.91 \quad p_{virginica} = 0.98$$



Arboles de decisión

Medidas de impureza

Impureza de Gini

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

Mide la probabilidad de clasificar incorrectamente un elemento si se lo etiqueta al azar según la distribución del nodo.

- Si todas las observaciones son de la misma clase → nodo puro → Gini = 0
- Si las clases están mezcladas → Gini aumenta
- El algoritmo busca minimizar la impureza total luego del split

Entropía e Information Gain

$$H = -\sum p_i \log_2(p_i)$$

La entropía proviene de la teoría de la información y mide el

nivel de incertidumbre.

Entropía = 0 implica certeza total, mientras que entropía máxima representa máxima incertidumbre.

$$IG = H(\text{parent}) - \sum w_j H(\text{child}_j)$$

La ganancia de información mide cuánto "aprendemos" al dividir el nodo.

El árbol elige la variable que **más reduce la incertidumbre.**

Overfitting

El problema fundamental

Un árbol puede seguir dividiendo hasta que cada hoja tenga una sola observación. En ese punto, el error de entrenamiento = 0, pero el modelo no generaliza.

Consecuencia

El árbol aprende reglas demasiado específicas y memoriza ruido en lugar de patrones generales.

Regularización en Árboles: Pruning

Los árboles no se regularizan con penalizaciones matemáticas como otros modelos, sino controlando su estructura. Existen dos enfoques principales:

Pre-pruning

Se limita el crecimiento del árbol desde el principio:

- Profundidad máxima (`max_depth`)
- Mínimo de muestras para dividir (`min_samples_split`)
- Mínimo de muestras por hoja (`min_samples_leaf`)

Post-pruning

Se construye el árbol completo y luego se simplifica eliminando ramas que no aportan mejora significativa.

❗ Regularizar un árbol es limitar su capacidad de memorizar ruido y mejorar su capacidad de generalización.

Hiperparámetros

La implementación de árboles de decisión en `sklearn` ofrece múltiples parámetros para controlar el comportamiento del modelo:

```
DecisionTreeClassifier(  
    max_depth=5,           # Profundidad máxima del árbol  
    min_samples_split=10,  # Mínimo de muestras para dividir un nodo  
    min_samples_leaf=5,    # Mínimo de muestras en una hoja  
    criterion="gini",      # Criterio: "gini" o "entropy"  
    max_features=None      # Número de características a considerar  
)
```

01

`max_depth`

Controla la complejidad y profundidad del árbol

03

`criterion`

Forma de medir la calidad de la separación

02

`min_samples_leaf`

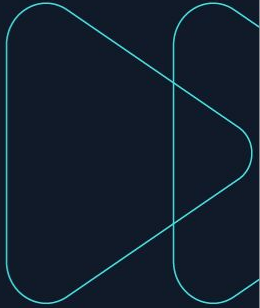
Evita hojas con muy pocas observaciones

04

`max_features`

Número de características consideradas en cada división

Random Forest

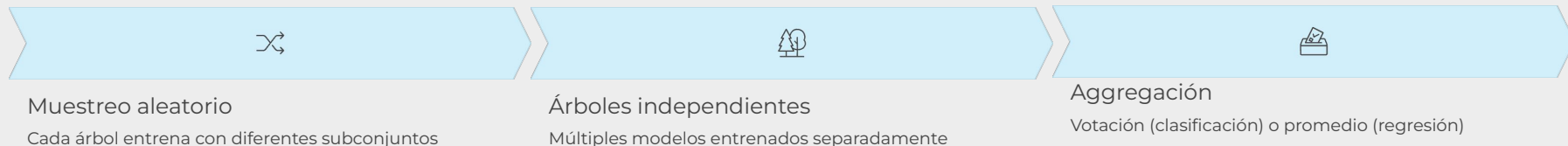


Random Forest

Surge como solución directa al principal problema de los árboles: su **alta varianza**. Un único árbol puede cambiar drásticamente si los datos cambian ligeramente. Random Forest reduce este problema combinando múltiples árboles entrenados de forma independiente.

Bagging

El algoritmo utiliza **bootstrap sampling**: cada árbol se entrena con una muestra aleatoria (con reemplazo) del dataset original. Esto genera diversidad entre los modelos.

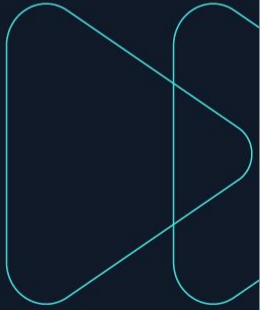


Desde un punto de vista estadístico, esto reduce la varianza porque **el promedio de múltiples estimadores independientes tiene menor varianza** que cada uno individualmente.

Selección Aleatoria de Variables

Además del muestreo de datos, en cada nodo el árbol solo evalúa un subconjunto aleatorio de variables. Esto evita que todos los árboles sean iguales y mejora la diversidad del ensemble, haciendo que el modelo sea más robusto y generalice mejor.

Máquinas de vectores de soporte



Máquinas de vectores de soporte (SVM)

A diferencia de los árboles, las SVM no construyen reglas, sino que trabajan desde una **perspectiva geométrica**. El objetivo es encontrar un hiperplano que separe las clases maximizando el margen.

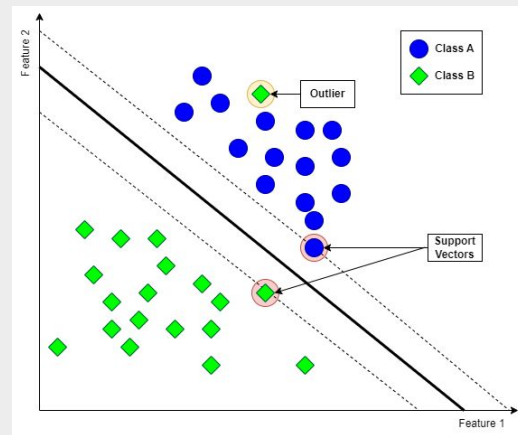
Vectores de Soporte

No todos los puntos importan. Solo los más cercanos al hiperplano determinan la solución. Esto tiene implicancias importantes:

- El modelo es eficiente
- La solución es robusta
- La complejidad depende de los puntos críticos

Hiperplano y Margen

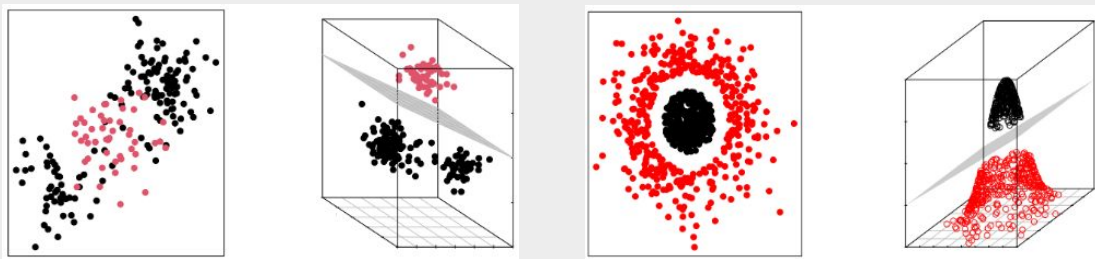
Un hiperplano es una generalización de una recta o plano a múltiples dimensiones. La SVM no busca cualquier separación, sino la que **maximiza el margen**. El margen es la distancia entre el hiperplano y los puntos más cercanos (vectores de soporte).



Máquinas de vectores de soporte (SVM)

¿Qué pasa si los datos no son separables linealmente?

- A veces, los datos no se pueden dividir con una simple línea recta. En estos casos, las **SVM utilizan una técnica llamada transformación de datos o kernel trick**.
- Este proceso permite que la **SVM transforme los datos a un espacio de mayor dimensión**, donde puede encontrar una separación clara entre los grupos.



Soft margin y Kernel

En datos reales, las clases no son perfectamente separables. Entonces se introduce una penalización:

Esto introduce un trade-off entre margen grande y errores de clasificación. El parámetro C controla ese equilibrio:

C alto

Modelo rígido, busca clasificar todo correctamente → posible overfitting

C bajo

Modelo flexible, permite más errores → mejor generalización

Kernel Trick:

Cuando los datos no son separables linealmente, SVM utiliza **kernels** para trabajar en espacios de mayor dimensión sin transformarlos explícitamente. Esto permite encontrar separaciones complejas sin costo computacional explícito.

SVM no cambia el modelo, cambia el espacio donde el problema se vuelve lineal.

Tipos de Kernels

Kernel lineal

Frontera recta en el espacio original

Kernel RBF

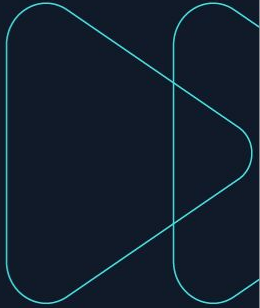
Fronteras curvas complejas

Kernel polinomial

Relaciones de orden superior

El kernel define la **forma del espacio donde el modelo separa**.

Regresión por vectores de soporte



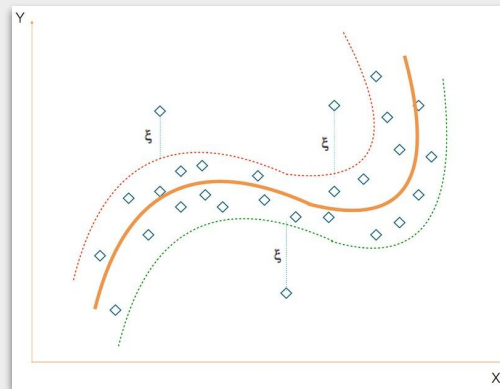
Regresión por vectores de soporte (SVR)

El Cambio de Paradigma: De SVM a SVR

Pregunta clave: ¿Y si en lugar de maximizar la distancia entre clases, intentamos que la mayor cantidad de puntos de datos queden dentro de nuestra "calle"?

La Idea Central de SVR: En lugar de separar, SVR busca ajustar una línea (hiperplano) a los datos de tal manera que la mayoría de los puntos queden dentro de un margen de error predefinido.

Acá el margen funciona al revés. No nos importa el error de los puntos que están dentro del margen. Solo nos penalizan los puntos que quedan fuera.



Regresión por vectores de soporte (SVR)

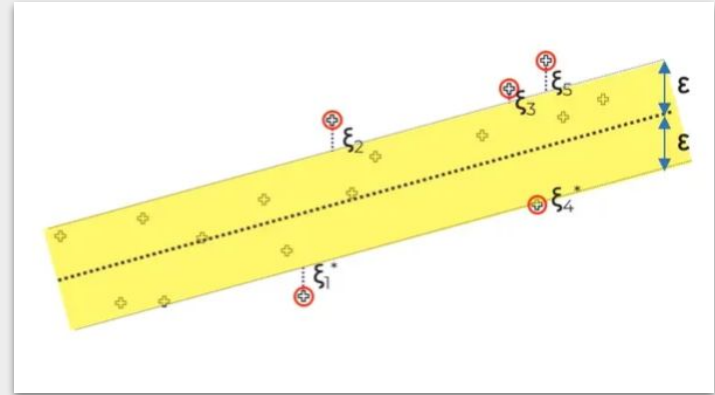
El Concepto Clave:

El "Tubo" o la **función de pérdida insensible a epsilon**

Margen de Tolerancia (ϵ): SVR define un "tubo" alrededor de la línea de regresión con un ancho de ϵ (épsilon) hacia arriba y hacia abajo.

Zona "Libre de Penalización": Cualquier punto de dato que caiga dentro de este tubo se considera una buena predicción.

Los Puntos que Importan: El modelo solo se preocupa por los puntos que caen fuera del tubo.



Regresión por vectores de soporte (SVR)

Hiperparámetros Clave a Configurar

kernel: El tipo de transformación a usar (linear, rbf, poly). Define la forma de la función de regresión.

C (Parámetro de Regularización): Controla el balance y minimizar el error de los puntos que quedan fuera.

- **C alto:** El modelo penaliza mucho los errores, pudiendo ajustarse demasiado a los datos de entrenamiento (sobreajuste).
- **C bajo:** El modelo es más tolerante a errores, creando un modelo más simple que podría no capturar la tendencia de los datos (subajuste).

epsilon (ϵ):

- Define el ancho del tubo donde no se penaliza el error.
- Un ϵ más grande permite que más puntos queden en el tubo, resultando en un modelo con menos vectores de soporte y potencialmente más simple.

Regresión por vectores de soporte (SVR)

Resumen - Ventajas y Desventajas de SVR

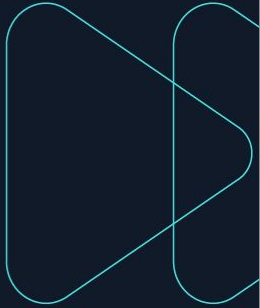
Ventajas:

-  Es muy efectivo en espacios de alta dimensión.
-  Es eficiente en el uso de memoria, ya que solo utiliza un subconjunto de puntos (los vectores de soporte).
-  Es versátil gracias a los diferentes kernels.

Desventajas:

-  No escala bien a conjuntos de datos muy grandes (el tiempo de entrenamiento puede ser largo).
-  La elección de los hiperparámetros (kernel, C, epsilon) es crucial y puede ser difícil de optimizar.
-  Los modelos resultantes son menos interpretables que un modelo de regresión lineal simple (es una "caja negra").

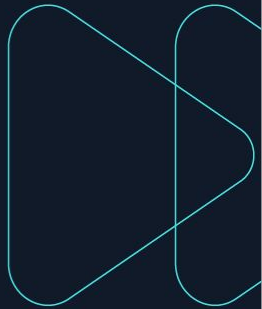
Resumen



Resumen de la clase

- Árboles de decisión
- Random forest
- SVM (clasificación)
- SVR (regresión)

¿Consultas?



¡Muchas gracias!

